

SENIOR SDET - AI TESTING

Complete Interview Playbook - 2026 Edition

LLM | Prompt Testing | RAG | Vector DB | DeepEval | RAGAS | LangChain | LangGraph | LangSmith | MCP | A2A | Agents
| Guardrails | Observability | Reliability | CI/CD

Interview probability markers included

MUST KNOW	Very high probability / core senior-level question
HIGH	Frequently asked follow-up or scenario
MEDIUM	Useful differentiator; role-specific or advanced

Target profile: Senior SDET / Lead AI QA / GenAI QA / Agentic AI Tester

Use this book for revision, mock interviews, scenario drills, and project explanation.

How the interview-probability tags were chosen

The priority tags are not guarantees. They are a practical ranking based on repeated requirements visible in recent 2026 Senior AI QA/SDET roles plus the core skills needed to explain an end-to-end AI quality strategy.

- Repeated MUST-KNOW themes: RAG, retrieval relevance/grounding, hallucination testing, golden datasets, LLM-as-a-Judge, DeepEval/RAGAS, agents/tool-calling, LangChain/LangGraph/LangSmith, vector databases, guardrails/prompt injection, CI/CD quality gates, trace-based assertions, Python/PyTest, reliability and production observability.
- HIGH themes: multi-agent handoffs, memory/state, responsible AI, data drift, streaming, cost/latency, fallback/model routing, MLOps/LLMOps.
- MEDIUM themes: A2A, multimodal/voice, fine-tuning validation and classical ML details unless the role specifically asks for them.

Contents

1. Interview positioning and project pitch
2. LLM fundamentals and prompt testing
3. Structured output, golden datasets and evaluation strategy
4. DeepEval, RAGAS, Promptfoo, TruLens and evaluator calibration
5. RAG, embeddings, vector DB and advanced retrieval
6. LangChain, LangGraph and LangSmith
7. MCP, A2A and tool-contract testing
8. Autonomous agents and trajectory testing
9. Multi-agent systems, memory, checkpoints and HITL
10. Guardrails, OWASP and adversarial testing
11. Observability, OpenTelemetry, latency, cost and reliability
12. Streaming, fallback, caching, AI API and performance
13. Data quality, drift, LLMOps/MLOps and CI/CD
14. Responsible AI, multimodal and classical ML basics
15. Real challenges and solutions
16. Senior scenario-based questions and answers
17. System-design and framework-design answers
18. Rapid revision sheets and formulas
19. Seven-day interview revision plan
20. Sources used for priority calibration

1. Interview positioning and project pitch

MUST KNOW | Q1. Tell me about your AI testing experience.

Hinglish concept: Start from automation experience, then show the AI quality layers.

Interview answer: I combine traditional SDET automation with AI-specific evaluation. I test LLM outputs, structured schemas, RAG retrieval and grounding, vector-search quality, MCP tools, autonomous and multi-agent workflows, guardrails, reliability, and observability. I use deterministic ground truth whenever possible, semantic evaluators where judgement is required, and dataset-level quality gates in CI/CD.

Follow-up / trap: Do not list tools without explaining how they fit into a release strategy.

MUST KNOW | Q2. Explain the AI testing project you built end to end.

Hinglish concept: Project answer ko architecture journey jaisa bolo, random files jaisa nahi.

Interview answer: I built an AI testing lab that progressed from LLM and RAG evaluation to MCP tool testing, Playwright-MCP browser agents, multi-agent handoffs, guardrails, observability, reliability and release gates. Each layer has deterministic validation, regression datasets and CI-friendly reports so a release must be correct, safe, reliable and observable.

Follow-up / trap: Interviewer may ask what was your hardest failure and how you fixed it.

HIGH | Q3. What is your core principle for testing non-deterministic AI systems?

Hinglish concept: Ek output par trust nahi; layered evaluation.

Interview answer: I decompose the system into deterministic and probabilistic parts. Schemas, tool names, document IDs, authorization, routing and final business state are validated deterministically; semantic qualities such as relevancy or faithfulness use calibrated evaluators. I then aggregate results over a representative dataset instead of trusting one run.

Follow-up / trap: Be ready to justify thresholds and evaluator calibration.

2. LLM fundamentals and prompt testing

MUST KNOW | Q4. How is LLM testing different from traditional software testing?

Hinglish concept: Same prompt ka wording badal sakta hai.

Interview answer: Traditional software is largely deterministic, while LLM outputs can vary while remaining semantically correct. Therefore exact string assertions are insufficient; I combine deterministic checks, semantic evaluation, repeated-run reliability tests and dataset-level thresholds.

Follow-up / trap: Interviewer may ask what can still be asserted exactly.

MUST KNOW | Q5. What is deterministic vs probabilistic evaluation?

Hinglish concept: Jo exact prove ho sakta hai usme LLM judge mat lao.

Interview answer: Deterministic evaluation has a clear expected answer such as a schema, tool name, ID, status or business state. Probabilistic evaluation judges qualities such as relevance, groundedness or tone. My rule is to prefer deterministic evidence whenever reliable ground truth exists.

Follow-up / trap: Good follow-up: can a deterministic metric still be wrong if ground truth is poor? Yes.

MUST KNOW | Q6. Why should you not use exact string matching for every LLM response?

Hinglish concept: Meaning same ho sakta hai, wording different.

Interview answer: Semantically equivalent answers can use different wording, so exact matching creates false failures. I assert critical facts and structure deterministically, then use semantic metrics for flexible natural-language portions.

Follow-up / trap: Mention exact matching is still appropriate for IDs, enums, JSON keys and policy-critical phrases when required.

MUST KNOW | Q7. How do you test prompts?

Hinglish concept: Prompt change = code change jaisa regression.

Interview answer: I treat prompts as versioned software artifacts. I test variable substitution, instruction hierarchy, few-shot examples, boundary inputs, injection resistance, output schema, regression against a golden dataset, and pairwise comparison between old and new prompt versions.

Follow-up / trap: Ask yourself: what dataset proves the new prompt is better?

HIGH | Q8. How do temperature and sampling settings affect testing?

Hinglish concept: Temperature low = stable testing; but robustness separately test karo.

Interview answer: Higher randomness generally increases output variability, so reliability tests should control or record inference settings. For deterministic regression I use stable settings where possible, while robustness tests intentionally sample multiple runs to estimate variance.

Follow-up / trap: Do not claim temperature zero guarantees identical outputs across every provider/runtime.

HIGH | Q9. How would you test a system prompt change?

Hinglish concept: A/B style offline comparison.

Interview answer: I run old and new versions on the same versioned dataset, compare deterministic business assertions and semantic scores, inspect regressions by intent and risk category, and block release on critical regressions. I also retain production failures as future regression cases.

Follow-up / trap: Mention pairwise evaluation and human review for ambiguous cases.

HIGH | Q10. What is prompt injection and why is it different from prompt testing?

Hinglish concept: Normal prompt quality vs adversarial prompt safety.

Interview answer: Prompt testing validates intended instructions; prompt-injection testing validates whether untrusted content can override or redirect those instructions. I test direct injection, indirect injection from retrieved/tool content, data exfiltration attempts and unauthorized action requests.

Follow-up / trap: Be ready to explain indirect prompt injection in RAG/tool outputs.

MEDIUM | Q11. What is ReAct?

Hinglish concept: Reason + Act workflow.

Interview answer: ReAct combines reasoning with actions so an agent alternates between deciding what to do and using tools. From QA perspective I validate the resulting action trajectory, tool selection, arguments, observations, stopping behavior and task outcome rather than trying to inspect hidden chain-of-thought.

Follow-up / trap: Do not claim you need access to hidden chain-of-thought to test an agent.

3. Structured output, golden datasets and evaluation strategy

MUST KNOW | Q12. How do you test structured LLM output?

Hinglish concept: Schema pass hona semantic correctness nahi hota.

Interview answer: I validate parsing, required fields, types, enums, nullability, extra fields and business constraints using JSON Schema or Pydantic. Then I separately evaluate whether the structured values are semantically correct.

Follow-up / trap: Example: valid category string can still be the wrong category.

MUST KNOW | Q13. What is a golden dataset?

Hinglish concept: Regression ka ground truth set.

Interview answer: A golden dataset is a curated, versioned set of representative inputs with expected outcomes such as labels, document IDs, tool calls, task states or safety decisions. It should include normal cases, edge cases, paraphrases, production failures and adversarial examples.

Follow-up / trap: Interviewer may ask how you avoid stale or biased golden data.

MUST KNOW | Q14. What is dataset-level quality gating?

Hinglish concept: 100 cases ka quality score, not one pass.

Interview answer: I aggregate per-case results into metrics such as completion rate, average relevance, safety failures or retrieval precision. Release decisions use thresholds and critical-failure rules rather than one response.

Follow-up / trap: Explain critical failures can override a good average.

HIGH | Q15. How do you choose evaluation thresholds?

Hinglish concept: Threshold ko green karne ke liye lower nahi karna.

Interview answer: I start from business risk and known labeled examples, analyze score distributions, calibrate against human or deterministic ground truth, then set thresholds that distinguish acceptable from unacceptable behavior. Thresholds should be versioned and reviewed, not chosen merely to make tests green.

Follow-up / trap: Mention separate thresholds by risk/intent if necessary.

HIGH | Q16. How do you prevent golden dataset leakage or contamination?

Hinglish concept: Test set ko training/tuning me leak mat hone do.

Interview answer: I version datasets, restrict direct use in prompt tuning where possible, maintain holdout cases, track provenance, deduplicate near-identical examples and periodically refresh from production failure patterns.

Follow-up / trap: For internal app evals, contamination can mean prompts were optimized too narrowly for known test cases.

HIGH | Q17. What is pairwise evaluation?

Hinglish concept: Old vs new output compare.

Interview answer: Pairwise evaluation compares two candidate outputs and asks which better satisfies the criteria. It is useful for prompt/model A/B decisions because relative judgement can be more stable than assigning absolute scores.

Follow-up / trap: Still calibrate evaluator bias and position/order effects.

4. DeepEval, RAGAS, Promptfoo, TruLens and evaluator calibration

MUST KNOW | Q18. What is DeepEval and where would you use it?

Hinglish concept: Pytest-friendly AI evaluation.

Interview answer: DeepEval is an LLM evaluation framework that integrates well with automated Python tests. I use it for metrics such as relevancy, faithfulness, GEval/custom criteria, tool correctness and test-case based regression, while keeping deterministic checks outside the judge when ground truth exists.

Follow-up / trap: Do not make every assertion an LLM metric.

MUST KNOW | Q19. What is LLM-as-a-Judge?

Hinglish concept: Judge bhi test subject hai.

Interview answer: LLM-as-a-Judge uses an evaluator model to score outputs against defined criteria. It is useful for subjective qualities, but the evaluator itself is probabilistic, so I calibrate it with known good/bad examples and track false positives and false negatives before trusting it as a hard release gate.

Follow-up / trap: Interviewer may ask what happens when judge disagrees with deterministic evidence.

MUST KNOW | Q20. Can an evaluator itself be wrong?

Hinglish concept: Evaluator ko blindly trust mat karo.

Interview answer: Yes. Evaluator models can be inconsistent, biased, threshold-sensitive or simply incorrect. I maintain evaluator calibration cases, compare against deterministic evidence/human labels, and downgrade an unreliable metric to diagnostic status instead of lowering thresholds to force green.

Follow-up / trap: This is a strong senior-level answer.

HIGH | Q21. What is GEval?

Hinglish concept: Custom semantic metric.

Interview answer: GEval is an LLM-based evaluation approach where custom criteria describe what good output means. I use it when built-in metrics do not capture a business-specific requirement such as completeness, policy adherence or professional quality.

Follow-up / trap: Keep criteria specific and test the evaluator with known examples.

HIGH | Q22. DeepEval vs RAGAS?

Hinglish concept: Tool selection use-case based.

Interview answer: DeepEval is a broader LLM testing/evaluation framework and is convenient in pytest-driven automation. RAGAS is strongly focused on RAG evaluation patterns. I choose based on architecture, metrics, CI integration and the team ecosystem rather than treating one as universally better.

Follow-up / trap: Know both; avoid claiming identical APIs/features.

HIGH | Q23. DeepEval vs LangSmith?

Hinglish concept: Evaluation framework vs evaluation+observability platform.

Interview answer: DeepEval is primarily an evaluation/testing framework; LangSmith combines tracing, datasets, experiments, evaluation and production observability. They can complement each other: DeepEval can run metric-driven tests while LangSmith manages traces and evaluation lifecycle.

Follow-up / trap: Good follow-up: when would you use LangSmith only?

HIGH | Q24. Where do Promptfoo and TruLens fit?

Hinglish concept: Framework landscape samjho.

Interview answer: Promptfoo is useful for prompt/model comparisons, automated evals and red-team style testing. TruLens focuses on evaluation/feedback and observability for LLM applications. For interviews I explain the capability category and when I would evaluate them, rather than claiming deep expertise with every framework.

Follow-up / trap: Senior answer should focus on evaluation strategy, not tool memorization.

MEDIUM | Q25. What is evaluator agreement and why does it matter?

Hinglish concept: Metric ka score reliable hai ya nahi?

Interview answer: Evaluator agreement measures consistency between automated evaluators and trusted labels or human reviewers. Low agreement means the metric may not be suitable as a release gate even if it produces precise-looking numeric scores.

Follow-up / trap: You can mention Cohen kappa or correlation if asked, but not necessary for most SDET roles.

5. RAG, embeddings, vector DB and advanced retrieval

MUST KNOW | Q26. Explain RAG architecture.

Hinglish concept: Retrieval + generation separate layers.

Interview answer: RAG retrieves external knowledge before generation. Documents are chunked and embedded into a vector/search index; the user query is transformed into a retrieval query, top candidates are fetched and optionally reranked, then selected context is provided to the LLM to generate a grounded answer.

Follow-up / trap: Interviewer may ask exactly where failures can occur.

MUST KNOW | Q27. How do you test a RAG system?

Hinglish concept: Correct context aya? Phir LLM ne sahi use kiya?

Interview answer: I separate retrieval and generation. Retrieval tests check expected document IDs, ranking, Precision@K, Recall@K, metadata filters, Top-K, duplicates and latency. Generation tests check faithfulness, answer relevancy, citation correctness and hallucination. End-to-end tests validate question to retrieved context to grounded answer.

Follow-up / trap: Always separate retrieval failure from generation failure.

MUST KNOW | Q28. What is faithfulness?

Hinglish concept: Context se supported?

Interview answer: Faithfulness measures whether claims in the generated answer are supported by the supplied/retrieved context. A response can be relevant to the question but still unfaithful if it invents unsupported facts.

Follow-up / trap: Be ready to contrast with answer relevancy.

MUST KNOW | Q29. What is answer relevancy?

Hinglish concept: Question ko answer kar raha hai?

Interview answer: Answer relevancy measures whether the response directly addresses the user question. A response can be grounded in context but still irrelevant or incomplete for the actual question.

Follow-up / trap: Contrast with correctness and faithfulness.

MUST KNOW | Q30. What is a vector database?

Hinglish concept: Semantic search ke liye embeddings store/search.

Interview answer: A vector database stores and searches high-dimensional embeddings for semantic similarity. In RAG, document chunks and queries are embedded, then nearest-neighbor search retrieves semantically relevant chunks, often combined with metadata filters.

Follow-up / trap: Vector DB retrieves; LLM generates.

MUST KNOW | Q31. What is an embedding?

Hinglish concept: Embedding model creates, DB stores/searches.

Interview answer: An embedding is a numerical representation of data where semantically similar content tends to be close in vector space. The embedding model creates the vectors; the vector database stores/indexes/searches them.

Follow-up / trap: Know dimension compatibility.

MUST KNOW | Q32. What are Precision@K and Recall@K?

Hinglish concept: Precision = retrieved me relevant kitne; Recall = relevant me retrieve kitne.

Interview answer: Precision@K is the proportion of the top K retrieved results that are relevant. Recall@K is the proportion of all expected relevant results that were successfully retrieved within the top K. Precision measures retrieval cleanliness; recall measures retrieval coverage.

Follow-up / trap: Be ready for a small numeric example.

HIGH | Q33. What is contextual precision vs contextual recall?

Hinglish concept: Ranking quality vs coverage.

Interview answer: Contextual precision focuses on whether relevant context is ranked ahead of irrelevant context, while contextual recall focuses on whether retrieved context contains enough information to support the expected answer.

Follow-up / trap: Metric implementations can differ by framework, so explain concept first.

MUST KNOW | Q34. What happens if the RAG answer is wrong? How do you debug it?

Hinglish concept: Sabse pehle: correct context retrieve hua?

Interview answer: First inspect whether the correct context was retrieved. If not, investigate chunking, embeddings, query transformation, filters, Top-K, reranking and index freshness. If correct context was retrieved, investigate prompt/context assembly, grounding and generation. This isolates retrieval failure from generation failure.

Follow-up / trap: This is one of the highest-value scenario answers.

HIGH | Q35. What is chunking and how do you test chunk size?

Hinglish concept: Chunk size empirical evaluation se choose karo.

Interview answer: Chunking splits large documents into retrievable units. Too-small chunks can lose context; too-large chunks add noise and token cost. I compare chunk size/overlap strategies on a golden retrieval dataset using retrieval metrics and answer quality instead of choosing values arbitrarily.

Follow-up / trap: Mention semantic chunking if role is advanced.

HIGH | Q36. What is metadata filtering?

Hinglish concept: Semantic + structured filter.

Interview answer: Metadata filtering constrains semantic search using structured attributes such as tenant, country, product, document type or date. I test correct filtering, unauthorized cross-tenant retrieval, missing metadata and interaction between filters and similarity ranking.

Follow-up / trap: Security/tenant isolation is a strong QA angle.

HIGH | Q37. What is hybrid search?

Hinglish concept: Vector + keyword.

Interview answer: Hybrid search combines semantic vector retrieval with lexical/keyword retrieval such as BM25. It can improve cases involving exact names, IDs or rare terms that embeddings may underweight. I evaluate hybrid vs vector-only retrieval on the same golden dataset.

Follow-up / trap: Be ready to explain why exact product codes benefit.

HIGH | Q38. What is reranking?

Hinglish concept: Retrieve many, rerank better.

Interview answer: Reranking takes an initial candidate set from the retriever and applies a stronger relevance model to reorder candidates before context selection. I test whether reranking improves relevant-document position without excessive latency or dropping required documents.

Follow-up / trap: Measure both quality gain and latency cost.

HIGH | Q39. What happens if the embedding model changes?

Hinglish concept: Embedding migration = retrieval regression mandatory.

Interview answer: The representation or vector dimension can change, so existing vectors may be incompatible or semantically inconsistent. I re-embed/reindex the corpus, validate index migration and rerun retrieval regression before release.

Follow-up / trap: Do not mix embeddings from incompatible models in the same index without a deliberate design.

HIGH | Q40. How do you test stale vectors and document updates?

Hinglish concept: Source change ke baad old knowledge retrieve nahi honi chahiye.

Interview answer: I update or delete a source document, verify the ingestion/index job updates the corresponding vectors, and assert old content is no longer retrieved. I also test duplicate versions, partial ingestion and eventual-consistency windows.

Follow-up / trap: Excellent production RAG scenario.

MEDIUM | Q41. Name common vector technologies and explain FAISS correctly.

Hinglish concept: Names + distinction.

Interview answer: Examples include Pinecone, Qdrant, Weaviate, Milvus, pgvector, OpenSearch/Azure AI Search vector capabilities and Chroma. FAISS is primarily a vector similarity search/indexing library rather than a full managed database platform.

Follow-up / trap: Tool depth depends on job stack.

6. LangChain, LangGraph and LangSmith

MUST KNOW | Q42. What is LangChain?

Hinglish concept: LLM app composition framework.

Interview answer: LangChain is a framework for composing LLM applications using models, prompts, tools, retrievers, structured outputs and agent abstractions. From a QA perspective I test components independently and then test the composed chain or agent end to end.

Follow-up / trap: Do not answer only "it connects LLMs."

MUST KNOW | Q43. How would you test a LangChain application?

Hinglish concept: Component + integration + E2E.

Interview answer: I test prompt/input contracts, structured outputs, retrievers, tool schemas, tool selection, callbacks/traces, error handling, retries and final business outcomes. For RAG I separate retrieval from generation; for agents I separate decisions from tool execution.

Follow-up / trap: Mention mocking expensive providers for fast tests.

MUST KNOW | Q44. What is LangGraph?

Hinglish concept: Stateful agent orchestration.

Interview answer: LangGraph is used to build stateful, graph-based agent workflows with nodes, edges, conditional routing, persistence and human-in-the-loop patterns. QA focuses on state transitions, routing conditions, checkpoint/resume behavior, loops and side-effect safety.

Follow-up / trap: Know Node, Edge, State, checkpoint, interrupt/resume.

HIGH | Q45. How do you test LangGraph state transitions?

Hinglish concept: Graph ko state machine jaisa test karo.

Interview answer: For each node I define preconditions, expected state mutation and allowed next edges. I test happy path, wrong routing, missing state, repeated execution, terminal states and loops. I also verify side effects are not duplicated after retry/resume.

Follow-up / trap: Strong follow-up: payment/email tool after checkpoint resume.

MUST KNOW | Q46. What is LangSmith?

Hinglish concept: Tracing + datasets + experiments + evals.

Interview answer: LangSmith is an observability and evaluation platform for LLM and agent applications. It supports traces, datasets, experiments, evaluators and production monitoring, helping teams debug model/tool/retrieval steps and compare regressions across versions.

Follow-up / trap: Know offline vs online evaluation.

MUST KNOW | Q47. What is offline vs online evaluation in LangSmith?

Hinglish concept: Pre-release dataset vs production traces.

Interview answer: Offline evaluation runs curated datasets before release to compare prompts, models or application versions. Online evaluation scores or monitors production traces after deployment to detect real-world regressions and feed failures back into datasets.

Follow-up / trap: Explain production failure -> dataset -> offline regression loop.

HIGH | Q48. What are Dataset, Experiment and Evaluator in LangSmith?

Hinglish concept: Dataset = cases, Experiment = run, Evaluator = score.

Interview answer: A Dataset contains test examples and optional expected outputs. An Experiment is a run of a target application/version over the dataset. Evaluators score the runs using code, heuristics, humans or LLM judges, allowing comparison between versions.

Follow-up / trap: Good for prompt/model regression explanation.

MUST KNOW | Q49. How would you debug a failed agent using LangSmith?

Hinglish concept: Trace se failure layer isolate karo.

Interview answer: I inspect the trace from original input through prompt/model calls, retriever results, tool selection, tool arguments, tool outputs, retries and final response. Then I classify whether the defect is planning, retrieval, grounding, tool execution, routing, latency or final generation.

Follow-up / trap: Mention correlation/trace IDs and redaction of sensitive data.

HIGH | Q50. LangSmith vs DeepEval?

Hinglish concept: Complementary tools.

Interview answer: LangSmith provides an evaluation and observability lifecycle with traces, datasets and experiments. DeepEval is highly convenient as a code-first testing/evaluation framework. I can use them together or choose one based on infrastructure and workflow.

Follow-up / trap: Avoid saying one is simply “better.”

HIGH | Q51. How do production traces become regression tests?

Hinglish concept: Prod bug -> golden dataset.

Interview answer: I identify representative failures, sanitize sensitive data, label the expected behavior, add them to a versioned evaluation dataset, reproduce them offline and keep them as permanent regression cases after the fix.

Follow-up / trap: This is a very strong mature-quality answer.

7. MCP, A2A and tool-contract testing

MUST KNOW | Q52. What is MCP?

Hinglish concept: Agent-tool integration standard.

Interview answer: Model Context Protocol standardizes how AI applications/agents discover and interact with external tools and context providers. For testing, I validate tool discovery, runtime schemas, arguments, authorization, execution errors and business outcomes independently of the LLM.

Follow-up / trap: Runtime tool schema is the source of truth.

MUST KNOW | Q53. How do you test an MCP server/tool?

Hinglish concept: Tool ko independently test before agent.

Interview answer: I test discovery/listing, tool names, required and optional arguments, type validation, unknown fields, authorization, successful execution, error responses, idempotency where relevant and business behavior. Then I test the tool through the agent separately.

Follow-up / trap: Do not confuse protocol success with business success.

MUST KNOW | Q54. What is the difference between tool execution success and task completion?

Hinglish concept: Tool success != task success.

Interview answer: Tool execution success only proves the call ran. Task completion proves the intended business goal was achieved. A click can succeed on the wrong element, so final state and semantic target grounding must be validated separately.

Follow-up / trap: Highest-probability agent question.

HIGH | Q55. Why should MCP/runtime schema be treated as source of truth?

Hinglish concept: Docs version drift ho sakta hai.

Interview answer: Documentation, examples or client wrappers can drift across versions. Runtime-discovered schemas reveal the actual required/optional fields and types the server accepts, so tests should validate against them and catch compatibility drift.

Follow-up / trap: Great scenario if tool args suddenly fail after upgrade.

HIGH | Q56. How do you test MCP authorization and least privilege?

Hinglish concept: Prompt restriction enough nahi; server auth mandatory.

Interview answer: I verify the agent receives only required tools/scopes, unauthorized calls are rejected, destructive tools need stronger controls, credentials are not exposed in prompts/traces, and server-side authorization is enforced even if the model attempts an unsafe call.

Follow-up / trap: Mention audit logs and tenant isolation.

HIGH | Q57. What is A2A and how is it different from MCP?

Hinglish concept: MCP = agent-tool; A2A = agent-agent.

Interview answer: Conceptually, MCP focuses on agent/application interaction with tools and context, while A2A focuses on collaboration between independent agents. For A2A-style flows I test capability discovery, task lifecycle, message/handoff schema, identity, correlation, retries and cross-agent context integrity.

Follow-up / trap: A2A is still a more role-specific topic than MCP.

MEDIUM | Q58. How would you test protocol/version compatibility?

Hinglish concept: Protocol upgrade = contract regression.

Interview answer: I run contract tests against supported client/server versions, validate schema changes, unknown fields, optional capability negotiation and backward compatibility. I keep representative integration tests pinned to production versions and add upgrade smoke tests before rollout.

Follow-up / trap: Mention not hardcoding dynamic server data.

8. Autonomous agents and trajectory testing

MUST KNOW | Q59. How do you test an autonomous AI agent?

Hinglish concept: Final answer hi test nahi; trajectory bhi.

Interview answer: I validate task understanding, tool necessity, tool selection, argument grounding, execution, retries, trajectory, stopping behavior and final task completion. I also evaluate safety, latency and reliability across repeated runs and a golden scenario dataset.

Follow-up / trap: List layers confidently.

MUST KNOW | Q60. What is agent trajectory testing?

Hinglish concept: Path to result matters.

Interview answer: Trajectory testing evaluates the ordered sequence of decisions and actions used to reach the result. I check tool choice, order, arguments, unnecessary steps, loops, retries and whether the trajectory achieves the goal safely and efficiently.

Follow-up / trap: A correct final state can still hide risky behavior.

MUST KNOW | Q61. What is element/tool grounding?

Hinglish concept: Valid ref != correct semantic target.

Interview answer: Grounding means the selected tool target or resource actually corresponds to the intended business entity in the current state. A syntactically valid reference is not enough; it must be the correct checkbox, customer, document or account.

Follow-up / trap: Use browser example: todo checkbox vs filter/global checkbox.

MUST KNOW | Q62. Decision retry vs execution retry?

Hinglish concept: Wrong AI decision vs transient tool failure.

Interview answer: A decision retry happens when the model chose an invalid/incorrect action and must re-plan. An execution retry happens when the action was valid but the external tool failed transiently. I track them separately because they indicate different quality problems.

Follow-up / trap: Do not blindly retry semantically wrong actions.

HIGH | Q63. What is decision efficiency?

Hinglish concept: Success ke saath efficiency bhi.

Interview answer: Decision efficiency can be measured as accepted decisions divided by total decision attempts. It identifies agents that eventually succeed but require many corrections, which affects latency, cost and reliability.

Follow-up / trap: Metric formula can vary; define yours explicitly.

HIGH | Q64. How do you detect a stuck loop?

Hinglish concept: Same action repeat + no progress.

Interview answer: I build a signature from goal/subgoal, tool and normalized arguments, then detect repeated identical or non-progressing actions beyond a threshold. I also compare state progress so repeated actions with no state change fail the run.

Follow-up / trap: More robust than only counting steps.

HIGH | Q65. Why not silently correct bad LLM arguments?

Hinglish concept: Test harness ko agent mat bana do.

Interview answer: Silently fixing semantic mistakes hides the quality defect and makes the framework look healthier than the agent. I reject invalid decisions, provide structured feedback for re-planning and measure the correction as part of decision efficiency.

Follow-up / trap: This is a very strong interview talking point.

HIGH | Q66. How do you test stopping behavior?

Hinglish concept: Goal complete = STOP.

Interview answer: I assert the agent stops when the goal is complete, does not continue taking unnecessary actions, respects a maximum-step guard and fails safely when completion cannot be proven.

Follow-up / trap: Excessive agency risk connects here.

HIGH | Q67. How do you evaluate task completion deterministically?

Hinglish concept: Agent claim != proof.

Interview answer: I inspect authoritative final state such as browser DOM/accessibility snapshot, API/database state or expected business record rather than relying on the agent saying “done.”

Follow-up / trap: Important when judge metrics disagree.

9. Multi-agent systems, memory, checkpoints and HITL

MUST KNOW | Q68. What is different about testing a multi-agent system?

Hinglish concept: Agent + handoff + orchestration.

Interview answer: I test each agent plus the contracts between them: routing, task ownership, dependencies, handoff schema, context preservation, error propagation, loop detection and overall task completion. Integration failures are distinct from answer-quality failures.

Follow-up / trap: Recent SDET roles explicitly call out multi-agent workflows.

MUST KNOW | Q69. How do you test context preservation between agents?

Hinglish concept: Buy milk handoff me Buy coffee nahi hona chahiye.

Interview answer: I carry stable identifiers such as correlation ID, plan ID, original goal, task ID and business entity through handoffs and assert downstream agents receive unchanged critical context. I also test tampering, missing fields and stale context.

Follow-up / trap: Mention schema validation.

HIGH | Q70. How do you test agent routing?

Hinglish concept: Right task -> right agent.

Interview answer: I define allowed task-type-to-agent mappings and test correct routes, unsupported tasks, wrong assignments, fallback routes and authorization. Routing should be deterministic when business rules are deterministic.

Follow-up / trap: Distinguish routing failure from agent reasoning failure.

HIGH | Q71. How do you test agent handoff loops?

Hinglish concept: Ping-pong detection.

Interview answer: I track source agent, target agent, task ID and progress state. Repeated A-to-B/B-to-A handoffs without progress beyond a threshold are classified as loops and fail the quality gate.

Follow-up / trap: Include max-hop protection.

HIGH | Q72. What is human-in-the-loop and where should it be used?

Hinglish concept: High-risk action me human approval.

Interview answer: HITL pauses automation for human review/approval when risk is high or judgement is ambiguous, such as payments, destructive actions or sensitive decisions. I test approve, reject, timeout, modify-and-resume, duplicate approval and restart/resume behavior.

Follow-up / trap: Good link to LangGraph interrupts/checkpoints.

HIGH | Q73. How do you test checkpoint and resume?

Hinglish concept: Resume after crash without duplicate payment/email.

Interview answer: I persist state, interrupt after a known step, restart/resume and verify completed steps are not duplicated, state is restored, pending work continues correctly and side-effecting tools are idempotent or protected.

Follow-up / trap: Excellent senior scenario.

HIGH | Q74. How do you test agent memory?

Hinglish concept: Memory correctness + isolation + security.

Interview answer: I test what should be remembered, what must not persist, tenant/user isolation, expiration, update/forget behavior, conflicting facts and retrieval from long-term memory. I also test memory poisoning and privacy leakage.

Follow-up / trap: Know short-term/state vs long-term memory conceptually.

MEDIUM | Q75. What is planner-executor architecture?

Hinglish concept: Plan vs execute separation.

Interview answer: A planner decomposes the goal into tasks while executors perform specialized actions. QA validates plan correctness, dependency ordering, assignment, executor outputs, re-planning after failure and final goal completion.

Follow-up / trap: Good architecture pattern to explain.

10. Guardrails, OWASP and adversarial testing

MUST KNOW | Q76. How do you test prompt injection?

Hinglish concept: Attack dataset + earliest boundary block.

Interview answer: I maintain an adversarial dataset covering direct and indirect injection, system-prompt extraction, unauthorized tool requests, secret exfiltration, external navigation and malicious retrieved content. I assert attacks are blocked at the earliest appropriate boundary and track detection and false-positive rates.

Follow-up / trap: Direct vs indirect injection is a common follow-up.

MUST KNOW | Q77. Where should guardrails exist?

Hinglish concept: Multiple layers, not one prompt.

Interview answer: I use defense in depth: input validation, planner/routing rules, handoff validation, tool allowlists and server-side authorization, output validation, audit logging and human approval for high-risk actions. I do not rely only on a system prompt.

Follow-up / trap: Server-side enforcement is key.

MUST KNOW | Q78. How do you test unauthorized tool usage?

Hinglish concept: Block before MCP/tool execution.

Interview answer: I define least-privilege allowlists and scopes, attempt unauthorized tools/arguments, verify the call is blocked before execution, and assert no side effects occurred. I also test whether prompt injection can influence tool selection.

Follow-up / trap: Mention auth scope, not just tool-name allowlist.

MUST KNOW | Q79. What OWASP risks matter for LLM/agent testing?

Hinglish concept: OWASP terminology ko testing cases me map karo.

Interview answer: Key areas include prompt injection, sensitive information disclosure, improper output handling, excessive agency, system-prompt leakage, vector/embedding weaknesses, misinformation, unbounded resource consumption, tool misuse and memory/context poisoning. I translate each relevant risk into adversarial tests and controls.

Follow-up / trap: Exact OWASP list versions evolve; explain risk categories, not memorized numbering only.

HIGH | Q80. What is indirect prompt injection?

Hinglish concept: RAG/tool content me hidden instruction.

Interview answer: Malicious instructions arrive through untrusted external content such as retrieved documents, web pages, emails or tool outputs rather than directly from the user. I mark external content as data, validate downstream tool actions and test malicious knowledge-base/tool payloads.

Follow-up / trap: Important for agentic RAG.

HIGH | Q81. What metrics do you use for guardrails?

Hinglish concept: Detection high + false positives low.

Interview answer: I track attack detection rate, missed attacks, false-positive rate, critical safety violations and block location. A system that blocks every input is not good; safety and usability both matter.

Follow-up / trap: Segment metrics by attack family.

HIGH | Q82. How do you prevent secrets from leaking through traces?

Hinglish concept: Observability itself can leak data.

Interview answer: I minimize recorded payloads, redact or hash sensitive fields, avoid storing raw credentials/prompts where unnecessary, enforce access controls and retention policies, and test tracing pipelines with synthetic secrets.

Follow-up / trap: Good Responsible AI/security intersection.

HIGH | Q83. How do you test excessive agency?

Hinglish concept: Agent ko extra freedom nahi.

Interview answer: I verify the agent cannot expand scope beyond the user-authorized goal, cannot invoke unnecessary high-risk tools, stops after completion, respects spending/action limits and requires confirmation for destructive actions.

Follow-up / trap: Connect to least privilege and stop conditions.

11. Observability, OpenTelemetry, latency, cost and reliability

MUST KNOW | Q84. Why is observability important for agents?

Hinglish concept: PASS hua, but how badly?

Interview answer: Two runs can both complete but differ dramatically in retries, tool failures, latency and cost. I trace the execution hierarchy and make operational quality part of the release gate, not just final functional correctness.

Follow-up / trap: Great senior-level distinction.

MUST KNOW | Q85. What do you trace in an agent system?

Hinglish concept: Orchestrator -> agent -> tool spans.

Interview answer: I capture trace/span IDs, parent-child relationships, stage/tool names, duration, status, retries, tool errors and selected safe attributes. Where available I also capture token usage and cost while redacting sensitive content.

Follow-up / trap: Do not log secrets blindly.

HIGH | Q86. What is OpenTelemetry and why is it relevant?

Hinglish concept: Vendor-neutral observability standard.

Interview answer: OpenTelemetry provides vendor-neutral standards for traces, metrics and logs. In GenAI systems I can map LLM calls, retrieval, agent/tool spans, latency, errors and usage to standardized telemetry and export it to the monitoring platform of choice.

Follow-up / trap: Role may mention LangSmith/Langfuse/Arize/OTel.

MUST KNOW | Q87. How do you find where latency is coming from?

Hinglish concept: Trace breakdown se bottleneck.

Interview answer: I compare end-to-end latency with nested spans: model decision time, retrieval time and tool/MCP time. If browser-agent time dominates while MCP calls are fast, the model/planning layer is likely the bottleneck; if MCP spans dominate, external execution is.

Follow-up / trap: Mention P95 for repeated runs.

HIGH | Q88. What latency metrics do you track?

Hinglish concept: P95 important.

Interview answer: I track end-to-end latency, time to first token for streaming, per-model/retrieval/tool latency and percentiles such as P50/P95/P99. Averages alone can hide tail latency.

Follow-up / trap: For local LLMs set realistic environment-specific budgets.

HIGH | Q89. How do you test cost regression?

Hinglish concept: No fake cost metrics.

Interview answer: I record actual token/usage data when the provider exposes it, calculate cost per scenario or workflow, compare against a baseline and fail or flag excessive cost. If usage is unavailable I report it as unknown rather than inventing zero usage.

Follow-up / trap: Excellent integrity answer.

HIGH | Q90. How do you test AI reliability/flakiness?

Hinglish concept: Same case N runs.

Interview answer: I repeat representative scenarios multiple times and measure pass rate, output/trajectory variance, retries, latency variance and safety violations. Reliability is a distribution, not one green run.

Follow-up / trap: Be ready to explain stable vs stochastic settings.

HIGH | Q91. What is a reliability threshold?

Hinglish concept: Risk-based threshold.

Interview answer: It is an agreed minimum success rate over repeated executions, often segmented by scenario criticality. Critical workflows may require much higher reliability and zero safety failures even if less-critical conversational tasks tolerate more variance.

Follow-up / trap: Do not present one universal number.

12. Streaming, fallback, caching, AI API and performance

HIGH | Q92. How do you test streaming LLM responses?

Hinglish concept: Streaming = partial output lifecycle.

Interview answer: I validate HTTP/stream establishment, time to first token, chunk ordering, final assembled output, disconnect handling, client cancellation, malformed events, duplicate chunks, safety behavior mid-stream and graceful completion. A 200 status with a truncated stream is still a functional failure.

Follow-up / trap: TTFT is a common metric.

HIGH | Q93. How do you test model fallback?

Hinglish concept: Primary fail -> fallback safely.

Interview answer: I simulate primary-provider errors, timeouts, rate limits and quality failures, verify fallback triggers only under configured conditions, preserves request context, avoids retry storms and still meets quality/safety requirements.

Follow-up / trap: Also test no fallback for non-retryable errors.

HIGH | Q94. How do you test model routing?

Hinglish concept: Right request -> right model.

Interview answer: I create cases for simple, complex, sensitive and cost-constrained requests and assert the correct model/route is selected. I then verify routing changes do not degrade quality, cost or latency.

Follow-up / trap: Deterministic routing rules should be deterministic tests.

HIGH | Q95. How do you test semantic caching?

Hinglish concept: Cache can serve stale/wrong answer.

Interview answer: I test exact and semantic hits, false matches between similar-but-different queries, tenant/user isolation, invalidation after knowledge changes, TTL and whether cached responses still satisfy current policy.

Follow-up / trap: Security isolation is crucial.

MUST KNOW | Q96. How do you test an AI API?

Hinglish concept: Traditional API + AI semantics.

Interview answer: I combine normal API testing - auth, status codes, schema, negative inputs, timeout/rate-limit behavior - with AI-specific checks such as structured output correctness, semantic quality, safety, consistency, token/cost and latency.

Follow-up / trap: Senior SDET answer should not abandon standard QA fundamentals.

HIGH | Q97. Which performance metrics matter for AI APIs?

Hinglish concept: Latency + throughput + cost + quality.

Interview answer: Average latency, P50/P95/P99, time to first token, throughput, error/timeout rate, token usage, cost per request and degradation under concurrency. I correlate performance with response quality because aggressive optimizations can reduce quality.

Follow-up / trap: Load tests should respect provider limits and cost.

MEDIUM | Q98. How do you test rate limiting and backoff?

Hinglish concept: 429 handling.

Interview answer: I exceed configured limits in a controlled environment, validate status/error contracts, retry-after/backoff behavior, jitter and maximum retries, and ensure clients do not create retry storms.

Follow-up / trap: Avoid blindly retrying non-idempotent actions.

13. Data quality, drift, LLMOps/MLOps and CI/CD

MUST KNOW | Q99. What AI tests should run on every PR vs nightly/release?

Hinglish concept: Fast PR, expensive live nightly/release.

Interview answer: PRs should run fast deterministic tests: schema, routing, tool contracts, guardrails, golden retrieval checks and quality aggregation. Expensive live LLM/MCP/browser/repeated-run evaluations belong in scheduled, manual or release pipelines unless latency/cost allows otherwise.

Follow-up / trap: This is a strong production answer.

MUST KNOW | Q100. How do you integrate AI evaluation into CI/CD?

Hinglish concept: Eval result -> quality gate -> release decision.

Interview answer: I produce machine-readable reports and exit codes from evaluation suites, define release thresholds, archive traces/reports, compare against baselines and block merges or releases on critical regression. I separate deterministic hard gates from calibrated probabilistic diagnostics where appropriate.

Follow-up / trap: Mention artifacts and trend history.

HIGH | Q101. What is data drift in AI applications?

Hinglish concept: Input/KB distribution changes.

Interview answer: Data drift means the production input or knowledge distribution changes compared with the data used to build/evaluate the system. I monitor intent mix, retrieval sources, schema/content distributions and quality metrics, then refresh datasets and baselines when drift is real rather than hiding regressions.

Follow-up / trap: Different from model drift.

HIGH | Q102. What is knowledge-base or retrieval drift?

Hinglish concept: RAG quality can change without LLM change.

Interview answer: The model may stay unchanged while indexed documents, metadata, chunking or embedding behavior changes, causing retrieval quality to degrade. I version ingestion/index configurations and run retrieval regression after changes.

Follow-up / trap: Very practical RAG debugging scenario.

HIGH | Q103. How do you manage test data for AI evaluation?

Hinglish concept: Golden data itself needs QA.

Interview answer: I version datasets, track provenance/labels, deduplicate, cover intents and risk categories, preserve hard production failures, maintain holdouts, sanitize PII and review stale cases. Dataset quality is part of test-framework quality.

Follow-up / trap: Recent roles emphasize data integrity and drift.

HIGH | Q104. What is LLMOps from a QA perspective?

Hinglish concept: Quality across model/prompt lifecycle.

Interview answer: LLMOps is the lifecycle for versioning and safely operating prompts, models, retrieval, datasets and evaluations. QA contributes offline evals, deployment gates, canary checks, production monitoring, drift detection and feedback loops from failures back into regression datasets.

Follow-up / trap: Connect to MLOps, but emphasize GenAI-specific artifacts.

HIGH | Q105. How would you validate a model/provider migration?

Hinglish concept: Provider swap is not just API contract.

Interview answer: Run old and new providers on the same representative dataset, compare quality, safety, structured-output compatibility, tool behavior, latency, cost and failure modes. Include fallback/timeout/rate-limit behavior before canary rollout.

Follow-up / trap: Pairwise + deterministic regression useful.

MEDIUM | Q106. What is canary evaluation for an AI feature?

Hinglish concept: Small production rollout with metrics.

Interview answer: A small fraction of traffic receives the new model/prompt/workflow while quality, safety, latency and cost are compared with the control. Automatic rollback or manual stop criteria should be defined before rollout.

Follow-up / trap: High-risk domains need stronger approvals.

14. Responsible AI, multimodal and classical ML basics

HIGH | Q107. What Responsible AI areas should an SDET test?

Hinglish concept: Policy -> concrete tests.

Interview answer: Bias/fairness, privacy and PII handling, harmful/toxic content, transparency, auditability, explainability where required, accessibility, retention and user-consent requirements. I convert policy requirements into explicit datasets and measurable checks.

Follow-up / trap: Do not give vague ethics-only answer.

HIGH | Q108. How do you test PII leakage?

Hinglish concept: Synthetic secret/PII canaries.

Interview answer: I seed synthetic PII, test prompts/retrieval/tool outputs and logs/traces, verify masking/redaction and authorization, and ensure cross-user/tenant data cannot leak. I also test that safeguards do not over-redact legitimate non-sensitive content.

Follow-up / trap: Check observability/storage too.

MEDIUM | Q109. How do you test multimodal AI?

Hinglish concept: Each modality layer independently + E2E.

Interview answer: I separate modality-specific preprocessing from model reasoning. For images I test image quality, OCR/object/content understanding, text-image consistency and unsafe inputs. For audio I test transcription, intent, noise/accent robustness, latency and generated speech quality.

Follow-up / trap: Role-specific but increasingly useful.

MEDIUM | Q110. What is Word Error Rate for voice AI?

Hinglish concept: ASR metric.

Interview answer: WER measures speech-recognition transcription errors based on substitutions, deletions and insertions relative to reference text. It is useful but should be combined with intent/task success because minor transcription errors may not affect business outcome.

Follow-up / trap: Do not use WER alone for conversational quality.

MEDIUM | Q111. What classical ML metrics should an AI tester know?

Hinglish concept: GenAI role me basics enough unless ML-heavy.

Interview answer: At minimum precision, recall, F1, accuracy, confusion matrix, ROC-AUC concepts and class imbalance. These remain relevant for classifiers, safety filters, routing models and hybrid AI pipelines.

Follow-up / trap: Know precision/recall trade-off.

MEDIUM | Q112. RAG vs fine-tuning?

Hinglish concept: Knowledge retrieval vs model adaptation.

Interview answer: RAG supplies external knowledge at runtime and is easier to update without changing model weights. Fine-tuning changes model behavior/weights and is useful for style, task adaptation or learned behavior. They solve different problems and can be combined.

Follow-up / trap: Interviewer may ask how testing differs after fine-tuning.

15. Real challenges and solutions

Challenge 1: LLM paraphrase caused an exact-string test failure [MUST KNOW]

Symptom: Semantically correct output failed automation.

Root cause: The test asserted exact text on a probabilistic natural-language response.

Solution: Move critical facts/schema to deterministic assertions and use semantic evaluation for flexible wording.

How to validate: Known paraphrases pass; genuinely incorrect facts still fail.

Challenge 2: LLM judge marked a deterministically successful agent as failed [MUST KNOW]

Symptom: Browser final state proved completion but judge score was low.

Root cause: The evaluator model produced a false negative.

Solution: Keep deterministic task-completion evidence as the hard gate; calibrate the judge with known good/bad cases and keep it diagnostic until reliable.

How to validate: Calibration set shows acceptable agreement before promotion to hard gate.

Challenge 3: RAG answer was wrong even though the LLM was capable [MUST KNOW]

Symptom: Generated answer hallucinated or missed policy.

Root cause: Correct source document was not retrieved.

Solution: Inspect expected vs actual document IDs, chunking, embeddings, filters, Top-K and reranking before tuning the generation prompt.

How to validate: Retrieval metrics recover and answer becomes grounded.

Challenge 4: Correct document was retrieved but answer was still wrong [MUST KNOW]

Symptom: Retriever looked healthy but response contradicted context.

Root cause: Generation/context assembly failure.

Solution: Inspect supplied context, prompt hierarchy, truncation, citations and faithfulness; add deterministic claim checks where possible.

How to validate: Faithfulness improves while retrieval metrics remain stable.

Challenge 5: Agent clicked the wrong UI element with a valid MCP ref [MUST KNOW]

Symptom: Tool call succeeded technically but business state was wrong.

Root cause: Validator checked tool/type but not semantic target grounding.

Solution: Resolve the intended element from the current snapshot/state and reject mismatched dynamic refs before execution.

How to validate: Wrong ref is blocked and corrected through decision retry.

Challenge 6: Dynamic Playwright MCP reference format changed [MUST KNOW]

Symptom: Parser worked for e21 but failed for f1e37-style refs.

Root cause: Hardcoded ref regex assumed one format.

Solution: Parse the runtime snapshot generically and never hardcode a specific ref prefix.

How to validate: Regression tests cover old/new dynamic formats.

Challenge 7: Bad agent action was retried as an execution failure [MUST KNOW]

Symptom: Same wrong MCP action executed twice.

Root cause: Decision validation was too weak, so a semantic error reached the tool layer.

Solution: Validate tool target/arguments before execution; use decision retry for semantic mistakes and execution retry

only for transient tool failures.

How to validate: Telemetry shows rejected decision rather than duplicate failed executions.

Challenge 8: One scenario exception aborted the entire golden dataset [MUST KNOW]

Symptom: Later scenarios never ran and no complete release report was produced.

Root cause: Dataset runner propagated per-case exceptions globally.

Solution: Isolate each scenario, convert execution exceptions into failed evaluations, continue remaining cases and fail release based on aggregate report.

How to validate: All cases execute and failed scenarios appear in final report.

Challenge 9: Agent succeeded but required too many decisions [MUST KNOW]

Symptom: Final state passed, but latency/cost were poor.

Root cause: Agent had low decision efficiency and repeated corrections.

Solution: Track accepted/total decisions, retry overhead and trajectory quality; enforce efficiency/latency thresholds in addition to completion.

How to validate: Task still passes functionally but quality gate flags inefficient runs.

Challenge 10: Prompt injection attempted to force an unauthorized tool [MUST KNOW]

Symptom: User/retrieved text instructed agent to call a powerful tool.

Root cause: Prompt-only controls were insufficient.

Solution: Use input/handoff guards plus deterministic tool allowlists and server-side authorization before MCP execution.

How to validate: Unauthorized call never reaches underlying tool and no side effect occurs.

Challenge 11: Production quality dropped after a prompt/model change [MUST KNOW]

Symptom: Unit tests passed but real users saw regressions.

Root cause: Offline dataset missed production distribution/failure cases.

Solution: Inspect production traces, add representative failures to the golden dataset, compare old/new experiments and add online monitoring/canary gates.

How to validate: Reproduced failures are fixed and become permanent regression cases.

Challenge 12: Agent resumed after a crash and duplicated a side effect [MUST KNOW]

Symptom: Email/payment/tool call happened twice.

Root cause: Checkpoint resume replayed a previously completed side-effecting step.

Solution: Persist step completion, use idempotency keys or transactional safeguards, and test resume/retry semantics explicitly.

How to validate: Crash/restart test produces exactly one side effect.

Challenge 13: Semantic cache returned a plausible but wrong response [MUST KNOW]

Symptom: Similar query hit the wrong cached answer.

Root cause: Cache similarity threshold/context keying was too broad.

Solution: Include tenant/context/version in cache key, tighten semantic threshold and test near-neighbor queries with different expected answers.

How to validate: False semantic hits are rejected while valid hits remain.

Challenge 14: RAG quality changed after embedding migration [MUST KNOW]

Symptom: Search still worked but ranking quality fell.

Root cause: New embedding representation changed retrieval behavior.

Solution: Re-embed/reindex consistently, benchmark on golden retrieval dataset and tune search/reranking thresholds.

How to validate: Precision/Recall and latency meet migration gates.

Challenge 15: Observability report showed zero token cost even though model ran [MUST KNOW]

Symptom: Dashboard underreported usage.

Root cause: Integration did not expose token usage but code defaulted unknown values to zero.

Solution: Represent unavailable usage as None/unknown and only calculate cost from actual provider usage records.

How to validate: Report distinguishes unavailable from genuine zero.

16. Senior scenario-based questions and answers

MUST KNOW | Scenario 1: A RAG answer is wrong. What is the first thing you check?

Strong answer: I first check whether the correct context/document was retrieved. If not, I investigate retrieval; if yes, I investigate generation/grounding. This prevents wasting time tuning the LLM for a retriever defect.

Senior signals: Layer isolation: retrieval vs generation.

MUST KNOW | Scenario 2: The agent says “done” but the tool trace looks fine. How do you prove completion?

Strong answer: I inspect authoritative final business state such as DOM/accessibility state, API/database record or workflow status. The agent statement and successful tool calls are supporting evidence, not proof.

Senior signals: Authoritative state beats self-report.

MUST KNOW | Scenario 3: The agent eventually succeeds after five wrong actions. Is it a pass?

Strong answer: Functional completion may pass, but agent-quality should fail or warn because decision efficiency, retry overhead, latency or safety thresholds were violated. I keep correctness and operational quality as separate gates.

Senior signals: Completion and quality are separate dimensions.

MUST KNOW | Scenario 4: A DeepEval/LLM judge score is 0 but deterministic evidence proves success. What do you do?

Strong answer: I do not lower the threshold just to pass. I investigate/calibrate the evaluator, keep deterministic ground truth as the hard gate and use the judge diagnostically until it demonstrates reliable agreement.

Senior signals: Evaluator testing mindset.

MUST KNOW | Scenario 5: A new prompt improves average score but breaks one critical banking workflow. Release?

Strong answer: No. Critical-case failures can override aggregate averages. I use risk-weighted or critical zero-tolerance gates so a high average cannot hide a severe business regression.

Senior signals: Critical failures override averages.

MUST KNOW | Scenario 6: Vector search returns relevant documents but the best one is ranked fifth. What do you test?

Strong answer: I inspect Precision/ranking, similarity strategy, hybrid retrieval and reranking. I compare before/after rankings on a golden set and measure whether reranking improves top positions without losing recall or increasing latency too much.

Senior signals: Ranking quality + performance trade-off.

MUST KNOW | Scenario 7: The model did not change, but RAG quality suddenly dropped. What could be wrong?

Strong answer: Knowledge-base content, ingestion, chunking, metadata, embedding/index version, filters, reranking or data distribution may have changed. I compare retrieval traces and index/version metadata before blaming the LLM.

Senior signals: RAG drift without model change.

MUST KNOW | Scenario 8: A prompt injection is hidden inside a retrieved PDF. How do you test this?

Strong answer: Treat retrieved content as untrusted data, seed a document containing malicious instructions, verify the agent does not reinterpret it as privileged instruction, and ensure any resulting unauthorized tool call is blocked by deterministic policy.

Senior signals: Indirect injection + defense in depth.

MUST KNOW | Scenario 9: Your agent has a delete tool. How do you make it safe?

Strong answer: Use least privilege, server-side authorization, constrained arguments, scope/business validation, explicit user confirmation or HITL for destructive actions, audit logs and adversarial tests. The model alone should never be the authorization boundary.

Senior signals: Server-side controls.

MUST KNOW | Scenario 10: A LangGraph workflow crashes after step 4. What do you test on resume?

Strong answer: State restores correctly, completed nodes are not duplicated, side-effecting tool calls are not replayed, pending work resumes from the right node, correlation IDs persist and terminal conditions remain correct.

Senior signals: Checkpoint/resume idempotency.

MUST KNOW | Scenario 11: Production users report bad answers that were never in your test set. What is your response?

Strong answer: Trace the failures, sanitize and label representative cases, add them to a versioned offline dataset, reproduce them, fix the system and keep the cases permanently as regressions.

Senior signals: Production feedback loop.

MUST KNOW | Scenario 12: Your LangSmith trace shows 60 seconds total, but MCP tools only consume 2 seconds. What is likely slow?

Strong answer: The remaining time is likely in model/planning/reasoning or orchestration. I inspect model-call and agent-decision spans, then compare repeated-run percentiles before optimizing.

Senior signals: Trace-based bottleneck isolation.

MUST KNOW | Scenario 13: The same prompt passes 8/10 times. Is that acceptable?

Strong answer: It depends on business risk. I report 80% reliability plus failure patterns and set a risk-based threshold. Critical transactional workflows generally require much higher reliability and zero unsafe outcomes.

Senior signals: Reliability is distribution + risk.

MUST KNOW | Scenario 14: A cached answer is outdated after a policy update. What failed?

Strong answer: Cache invalidation/versioning. I key cache entries on relevant knowledge/prompt/model versions or invalidate on content changes and add stale-cache regression tests.

Senior signals: Caching part of AI quality.

MUST KNOW | Scenario 15: Primary LLM provider times out and fallback answers differently. What do you test?

Strong answer: Fallback trigger rules, context preservation, output schema/tool compatibility, quality/safety thresholds, retries, latency and whether non-idempotent side effects can be duplicated during failover.

Senior signals: Resilience + semantic compatibility.

MUST KNOW | Scenario 16: A model migration reduces cost by 40% but P95 latency rises and retrieval answer quality falls slightly. How do you decide?

Strong answer: Compare against pre-agreed quality, safety, latency and cost SLOs by scenario/risk. Cost benefit does not justify crossing critical quality or latency thresholds; use weighted trade-offs and canary data rather than one metric.

Senior signals: Multi-objective release decision.

MUST KNOW | Scenario 17: How would you test an AI-generated JSON response that is valid but wrong?

Strong answer: Schema validation proves syntax/contract only. I then validate field values against deterministic business rules or golden labels and use semantic evaluators only for fields that genuinely require judgement.

Senior signals: Contract vs semantic correctness.

MUST KNOW | Scenario 18: An agent keeps selecting “Completed” filter instead of the todo checkbox. How do you fix testing?

Strong answer: Add semantic element grounding from the current accessibility snapshot: identify the checkbox associated with the specific todo and reject any other valid ref before tool execution. Feed the correction back as a decision retry.

Senior signals: Dynamic state grounding.

MUST KNOW | Scenario 19: A tool call returned is_error=True. Should you always retry?

Strong answer: No. Retry only if the action was valid and the failure is classified as transient/retryable. Semantic errors, auth failures or non-idempotent uncertain side effects require different handling.

Senior signals: Safe retry classification.

MUST KNOW | Scenario 20: How do you test cross-tenant vector retrieval?

Strong answer: Create similar content for two tenants, query as each tenant, verify metadata/auth filters prevent cross-tenant chunks from appearing, and inspect logs/traces for leaked identifiers.

Senior signals: Security + retrieval.

HIGH | Scenario 21: How would you test a human-approval step?

Strong answer: Test approve, reject, modify, timeout, cancellation, duplicate response, unauthorized approver, restart/resume and exactly-once execution of the downstream action.

Senior signals: HITL is a workflow, not just a button.

HIGH | Scenario 22: Your evaluation score improved after adding lots of easy cases. Is the system actually better?

Strong answer: Not necessarily. Dataset composition can inflate averages. I track coverage by intent/risk, preserve hard cases, use stratified metrics and compare on a stable holdout dataset.

Senior signals: Dataset gaming/quality.

HIGH | Scenario 23: A safety guard blocks legitimate requests containing the word “password.” What metric exposes this?

Strong answer: False-positive rate on a benign dataset. I test both attack detection and benign acceptance, then refine patterns/context so safety does not simply block everything.

Senior signals: Safety + usability.

HIGH | Scenario 24: The agent chooses the correct tool but invents an argument that is absent from the current state. What should happen?

Strong answer: Validate against runtime schema and current authoritative state, reject the decision, provide structured feedback for re-planning and record the rejected decision. Do not silently fabricate or repair business-semantic arguments.

Senior signals: Grounding + honest evaluation.

HIGH | Scenario 25: How would you test a streaming response that starts safely but later emits sensitive data?

Strong answer: Inspect/scan stream chunks or policy checkpoints, ensure the client can terminate/redact unsafe output, validate final assembled content and audit whether sensitive partial content was exposed before cancellation.

Senior signals: Mid-stream safety.

HIGH | Scenario 26: How would you test tool/version drift after an MCP upgrade?

Strong answer: Rediscover runtime tools/schemas, run contract tests for required/optional fields and capabilities, execute representative tools, then run agent integration tests to detect prompt/schema assumptions that no longer match runtime behavior.

Senior signals: Runtime contract is source of truth.

MUST KNOW | Scenario 27: How do you decide which AI tests belong in PR CI?

Strong answer: Put deterministic, fast, low-cost contract/guardrail/retrieval tests in PR CI. Run expensive real-model, browser/MCP, reliability and performance suites scheduled or pre-release, while keeping at least a minimal live smoke test if practical.

Senior signals: Fast feedback without losing live coverage.

MUST KNOW | Scenario 28: How would you test a multi-agent handoff where the downstream agent receives the wrong customer ID?

Strong answer: Validate handoff schema plus immutable business identifiers/correlation IDs before execution, reject tampered/mismatched context and assert no downstream side effect occurs.

Senior signals: Context integrity.

HIGH | Scenario 29: A retrieval evaluator says contextual recall is perfect, but you know a required document is missing. What do you do?

Strong answer: Use deterministic expected document IDs/Recall@K as ground truth where available, investigate evaluator calibration, and do not weaken the deterministic gate because a probabilistic metric disagrees.

Senior signals: Deterministic retrieval ground truth.

MUST KNOW | Scenario 30: How would you explain a go/no-go release decision to leadership?

Strong answer: Summarize critical workflow completion, safety incidents, dataset pass rates, reliability, major regressions, latency/cost SLOs and known risks. Make the decision explicit: PASS, CONDITIONAL PASS or FAIL, with evidence and mitigation rather than raw metric dumps.

Senior signals: Senior ownership and communication.

17. System-design and framework-design answers

A. Design an AI testing framework

```
Test Data Layer
- Golden datasets
- Adversarial datasets
- Production failure corpus
- Ground truth / labels
|
Execution Layer
- LLM APIs / structured outputs
- RAG / vector retrieval
- MCP tools / APIs / browser
- Agents / multi-agent workflows
|
Evaluation Layer
- Deterministic assertions
- Retrieval metrics (P@K / R@K)
- DeepEval / RAGAS / LLM judges
- Safety / trajectory metrics
|
Observability Layer
- Traces / spans
- Latency / retries / failures
- Token usage / cost when available
|
Quality Gate Layer
- Per-case thresholds
- Dataset pass rates
- Critical-failure rules
- Baseline regression
|
CI/CD
- Fast PR gates
- Nightly/live evaluation
- Pre-release reliability/performance
- Reports and release decision
```

Interview answer: I would separate execution, evidence collection, evaluation and release policy. Deterministic ground truth remains the strongest signal for schemas, IDs, tool contracts and business state; probabilistic metrics are calibrated and used where semantic judgement is necessary. I would build fast PR gates and separate expensive live evaluations, then feed production failures back into versioned datasets.

B. Explain your project architecture in 90 seconds

Model answer: I started with LLM and structured-output tests, then built a RAG service and retrieval evaluation with deterministic Precision@K/Recall@K plus faithfulness and relevancy metrics. I exposed business capabilities through MCP and built an autonomous Playwright-MCP browser agent. I added pre-execution grounding, decision vs execution retry telemetry, stuck-loop detection and dataset-level quality gates. Then I introduced planner-browser-quality multi-agent handoffs, guardrails for prompt injection and unauthorized tools, and finally tracing, latency, reliability, drift and CI/CD release gates. The key design is layered evidence: I can identify whether a failure is retrieval, generation, planning, routing, tool execution, safety or infrastructure instead of treating the AI system as one black box.

18. Rapid revision sheets and formulas

Concept	Meaning	Remember
Faithfulness	Is the answer supported by context?	Grounding
Answer relevancy	Does the answer address the question?	Question alignment
Precision@K	Relevant retrieved / K	Retrieval cleanliness
Recall@K	Relevant retrieved / total relevant	Retrieval coverage
Embedding	Numerical semantic representation	Model creates vectors
Vector DB	Stores/indexes/searches vectors	Retriever, not generator
LangChain	LLM app composition	Prompt/model/retriever/tools
LangGraph	Stateful graph-based orchestration	State, edges, checkpoint, HITL
LangSmith	Tracing + datasets + experiments + eval	Offline + online evaluation
MCP	Agent/tool-context integration	Runtime schema
A2A	Agent-to-agent collaboration	Handoff/task lifecycle
Decision retry	Bad AI action -> re-plan	Agent quality
Execution retry	Valid action -> transient tool failure	Tool reliability
P95 latency	95% requests finish at/below this value	Tail latency
Attack detection rate	Blocked attacks / attack cases	Safety recall-like signal
False-positive rate	Benign cases incorrectly blocked / benign cases	Usability of guardrail

Top 25 questions to revise first

21. How is LLM testing different from traditional automation?
22. Deterministic vs probabilistic evaluation?
23. What is a golden dataset and dataset-level quality gate?
24. What is LLM-as-a-Judge and can the evaluator be wrong?
25. Explain RAG architecture and how you test it.
26. Faithfulness vs answer relevancy?
27. What is a vector DB and an embedding?
28. Precision@K vs Recall@K?
29. How do you debug a wrong RAG answer?
30. How do you test prompt changes / prompt regression?
31. What is LangChain?
32. What is LangGraph and how do you test state/checkpoints?
33. What is LangSmith? Offline vs online evaluation?
34. LangSmith vs DeepEval?
35. What is MCP and how do you test MCP tools?
36. Tool execution success vs task completion?
37. How do you test an autonomous agent?
38. What is trajectory and element/tool grounding?
39. Decision retry vs execution retry?
40. How do you test multi-agent handoffs and context preservation?
41. How do you test prompt injection and guardrails?
42. What metrics belong in an agent quality gate?
43. Why does observability matter and what do you trace?
44. How do you test reliability/flakiness?
45. How do you integrate AI evaluation into CI/CD?

19. Seven-day interview revision plan

Day	Focus
Day 1	LLM basics, deterministic/probabilistic evaluation, prompt testing, structured outputs, golden datasets. Practice project introduction.
Day 2	RAG, embeddings, vector DB, chunking, hybrid search, reranking, Precision@K/Recall@K and RAG debugging scenarios.
Day 3	DeepEval, RAGAS, LLM-as-a-Judge calibration, LangChain, LangSmith datasets/experiments/traces.
Day 4	LangGraph, MCP, autonomous agents, trajectory, grounding, decision/execution retries and task completion.
Day 5	Multi-agent handoffs, memory/checkpoint/HITL, guardrails, OWASP, prompt injection and adversarial datasets.
Day 6	Observability, OpenTelemetry, reliability/flakiness, streaming, fallback, caching, cost/latency, data drift and CI/CD.
Day 7	Only mock interviews: 90-second project pitch, 30 scenarios, framework-design answer, go/no-go release decision and weak areas.

20. Sources used for priority calibration

Priority markers are a practical interview-frequency estimate, not a statistical guarantee. Recent role descriptions reviewed while preparing this edition repeatedly emphasized the following areas: RAG/Vector DB, LLM evaluation, DeepEval/RAGAS, LangChain/LangGraph/LangSmith, agent/tool testing, prompt injection/guardrails, CI/CD, trace-based assertions, observability, drift and Python/PyTest.

- QualityAI - SDET AI ML Validation Engineer (30 Jul 2026) - <https://careers.quality-ai.com/job/Bangalore-SDET-AI-ML-Validation-Engineer-560045/57474844/>
- Tekion - Senior Software Development Test Engineer / Generative AI & LLM Evaluation - <https://in.linkedin.com/jobs/view/senior-software-development-test-engineer-at-tekion-corp-4452681795>
- Kobie Marketing - Lead AI QA Engineer - <https://jobs.lever.co/kobie/3280d456-5db2-4070-a7eb-d879c17c1393>
- Deutsche Telekom Digital Labs - Senior SDET / LLM multi-agent systems - <https://in.linkedin.com/jobs/view/senior-sdet-at-deutsche-telekom-digital-labs-4418051945>
- Fidelity International - AI Test Engineer / Test Lead - <https://in.linkedin.com/jobs/view/ai-test-engineer-test-lead-at-fidelity-international-4461561592>
- KPMG India - DEQ Senior AIML / LangChain LangGraph LangSmith - <https://in.linkedin.com/jobs/view/deq-senior-aiml-at-kpmg-india-4431350227>
- OnBoard - Senior AI Engineer, Observability - <https://job-boards.greenhouse.io/onboardmeetings/jobs/5836085004>

Final preparation rule: Do not try to memorize every sentence. Memorize the architecture, distinctions, debugging order, metrics, and 90-second project story. Then practice answering scenarios aloud.